agent-runtime 对话流水

服务端 Coding Agent。事实源是 append-only 的 events.jsonl，模型看到的 Messages 只是投影。组合根在 shared/deps.Build，循环本身无会话状态。

源码阅读 · agent-runtime · 2026-10-27

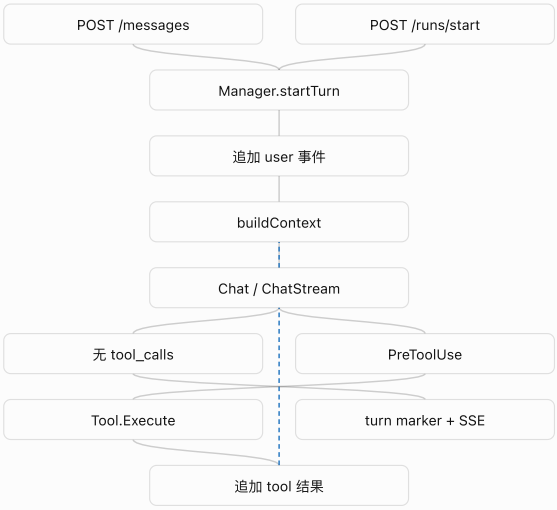
6	2	1	4
可替换层（对齐 dsh）	回合入口，汇合到 startTurn	正常终止：不再调工具	保护性终止码

读代码时抓住的四条不变量

每个 tool_call 必有配对结果；工具失败回注为普通结果；系统提示词和工具定义是稳定前缀，时间与 cwd 走状态栏；审批与 MCP 都经过同一条 Hook，不旁路。

一轮对话

左图是调用链，虚线是工具结果回到下一轮投影。右侧按阅读顺序展开每一步落在哪个函数。



1. HTTP 接入 | 2. 启动唯一回合 | 3. 用户消息落盘 | 4. 投影上下文 | 5. 调模型与恢复 | 6. 终止或执行工具 | 7. 钩子、审批、执行 | 8. 回注并推 SSE

1. HTTP 接入 `interfaces/httpapi/server.go`

Web UI 走兼容入口 POST /v1/sessions/{id}/messages，立即 202。控制面先 POST .../runs（Prepare，幂等、校验 payload_hash 与模型），再 POST .../runs/{runID}/start。两条路都进 Manager.startTurn。

分层与职责

层	路径	这一轮里做什么
入口	cmd/agent-runtime/main.go	加载配置、deps.Build、挂 HTTP、退出时关 MCP
组合根	internal/shared/deps	模型仓储、工具、钩子、Loop、Manager、RunService、Orchestrator
接口	interfaces/httpapi、webui	REST、SSE、内嵌控制台；鉴权与 1MiB 请求体上限
应用	application/agent、session、project	ReAct 循环、回合与 Run 生命周期、规划后 fork 编码
领域	domain/entity、port、plugin、derr	四角色消息、端口、注册表、哨兵错误
基础设	infrastructure/adapter/*	OpenAI、JSONL、bash/文件/搜索、审批、技能

基础设施	infrastructure/adapter/	OpenAI、JSONL、bash/文件/搜索、审批、技能、MCP、状态栏

依赖方向：interfaces 与 application 只依赖 domain/port；adapter 实现端口。application 不 import infrastructure。

建议阅读顺序

先把契约和一条主路径读完，再按需下钻工具实现。主路径大约 90 分钟；侧路每条 20–40 分钟。

顺序	文件	读到什么程度就停
1	README.md	产品形态、与 dsh 的层对应、已知边界。跳过已过时的缺口描述，以代码为准。
2	domain/port/ports.go	LLMPort、Tool、Hook、SessionRuntime、RunStore。这是循环的全部世界。
3	domain/entity/message.go	Message / Event 五种 Kind，分清谁进上下文、谁只广播。
4	shared/deps/deps.go Build	启动时注册了哪些工具和钩子，不必逐个打开工具文件。
5	httpapi/server.go	只看 postMessage、prepareRun、startRun、stream、resolveApproval。
6	session/manager.go	startTurn、AppendEvent、Messages、Approve。压缩先看注释。
7	agent/loop.go	通读。对照 loop_test.go 的 ReAct、幻觉工具、活性、预算、重试。
8	policy/policy.go PreToolUse	三档模式与强制审批。命令解析器等遇到 bash 风险再读。
9	storejsonl/store.go Append	seq 如何分配、fork 如何复制前缀。

主路径之后的侧路

先读	用到再读
session/run.go Prepare 与 Start 的幂等、	adapter/tools/bash.go：持久 shell、超时、